

APPENDIX A OPEN SOURCE CODE

The source code for *NEOviz* is divided into two parts: pre-processing, available at <https://github.com/fei0324/NEOviz/releases/tag/tvcg-2026>, and visualization, available at <https://github.com/OpenSpace/OpenSpace/releases/tag/research%2Fneoviz>.

APPENDIX B PRE-PROCESSING

The pre-processing is implemented with Python 3.11.5 on Ubuntu using Windows Subsystem for Linux (WSL).

A. Data Generation

The first step in the pre-processing of *NEOviz* is to generate data. The experts use the orbit determination software `find_orb` to generate state vectors of the asteroid over time. `find_orb` has a robust suite of orbit fitting techniques that include a variety of methods for initial orbit determination and full least-squares differential orbit correction [69]. Given a set of observations from `mpcq`, `find_orb` determines the best-fitting orbit, whose uncertainty is calculated as a covariance matrix during the least-squares optimization process. The orbit fitting process for each object is run iteratively over time, where every new submission includes the observations from previous submissions. This effectively mimics how each asteroid was observed in practice, and how our knowledge of its orbit evolves with further observation. For `find_orb` to reliably find an orbit for an asteroid, a few observations are required as a starting point. Therefore, it is important to select a starting time at a point where there have been a few observations. The result from the orbit determination process includes the observations, submission history, the best-fit orbit derived from these observations, and the time-varying uncertainty.

B. Propagation

Using the data generated from `find_orb`, the best-fitting orbit is propagated over time using the orbital integrator ASSIST (the ASSIST propagator), and the covariance matrix is reconstructed with a Monte Carlo sampling method. Using the best-fitting orbit, we then randomly sample variant seeds from the covariance matrix via Monte Carlo sampling. It is important to carefully consider the starting time used to generate variants. Optimally, the variant seeds should be generated close to a submission to avoid the uncertainty region being deformed by Keplerian shear or other forces.

The variant seeds are individually propagated forward in time using the ASSIST propagator. During the propagation, manual batching is performed in order to avoid memory issues. The batching occurs in the temporal domain, where the propagation is performed in small time intervals that are combined to the full time span. Due to a memory issue in a dependency library of the propagator, memory is managed by only activating the propagator while it is in use. These issues are not crucial if the user runs a shorter time interval with

a few time steps or uses a lower number of variant samples. However, if the user runs many time steps with many variants, such as 10,000 variants over several years with a sampling of once every minute, memory issues can occur.

To determine the time steps for the propagation, the number of whole days between the given start and stop times is calculated. The user can set the number of time steps per day, which is multiplied by the number of days to give the total number of time steps. A minimum of 80 time steps is enforced to reliably generate SPICE kernels [57], [58], which are used for impact analysis. The time span between the start and stop time is then equally divided into the total number of time steps. For some asteroids (such as Apophis), the user must propagate over long time spans (e.g., over several decades) to reach the time of interest for analysis. To address this, *NEOviz* supports non-uniform time ranges, allowing users to define an additional subrange within the overall time range with a user-specified, denser temporal sampling.

For *sectioned uncertainty tubes*, the time range of each submission is connected. To make the change apparent in the uncertainty tube, a time gap is introduced at the connecting edges. This time gap is specified by the user as a percentage of the length of the previous submission. For the figures in the paper, we typically use a time gap of 5%.

C. Ellipse Generation

When all variant orbits have been propagated, the potential positions of the asteroid for each time step are collected into a 3D point cloud. From the point cloud of each time step, a 2D plane of the uncertainty tube is generated. We define this 2D plane by using the median velocity direction (v_{median}) as the normal and the position of the best-fitting orbit at the same time as a reference point on the plane. Each point in the 3D point cloud is then projected onto this plane by following its velocity direction until it intersects the plane. Now that all points are on a 2D plane, the problem is reduced to a standard 2D setting by mapping the plane onto the XY-plane via a translation and rotation. The translation is the known point on the plane (the position of the best-fitting orbit at the same time), and the rotation matrix, R_{tot} , is calculated with eq. (1) where I is a 3×3 identity matrix, R is the rotation axis, and n_{xy} is the normal of the XY-plane (i.e. the Z-axis).

$$R_{tot} = I + K \sin a + KK(1 - \cos a), \text{ where}$$

$$K = \begin{pmatrix} 0 & -R_z & R_y \\ R_z & 0 & -R_x \\ -R_y & R_x & 0 \end{pmatrix} / |R| \quad (1)$$

$$\begin{aligned} R &= v_{median} \times n_{xy} \\ \sin a &= v_{median} \cdot n_{xy} \\ \cos a &= |R| \end{aligned}$$

To find the minimum-area enclosing ellipse of the points on the XY-plane, we use the MVEE [56] algorithm. The algorithm outputs three quantities: the fitted ellipse center relative to the

source place, the two semi-axis lengths a and b ordered largest first, and a 2×2 rotational matrix that gives corresponding axis directions in the plane. These outputs are directly used in the boundary sampling computation of the ellipse.

The boundary of the ellipse is sampled into a user-specified number of segments of equal arc length. Due to the asymmetric nature of the uncertainty region, it is not unusual for the ellipse to have high eccentricity. Unlike circles or spheres, where uniform parameterization produces evenly spaced samples, ellipses exhibit non-uniform spacing under such sampling. To avoid this, we enforce equal arc length sampling along the boundary. To do this, the full ellipse circumference, c , is calculated with eq. (2), where $E(m)$ is the complete elliptic integral of the second kind, and a and b are the lengths of the semi-major and semi-minor axes of the ellipse, respectively. In *NEOviz*, this integral is approximated by discretizing the interval $[0, \frac{\pi}{2}]$ into small segments $\Delta\theta$. The circumference is then divided into the number of user-specified sections, giving the desired arc length for each section. The boundary of the ellipse is sampled using this arc length.

$$\begin{aligned} c &= 4aE(m), \text{ where} \\ m &= 1 - \frac{b^2}{a^2} \\ E(m) &= \int_0^{\frac{\pi}{2}} \sqrt{1 - m \sin^2 \theta} d\theta \end{aligned} \quad (2)$$

D. Alignment

To construct the uncertainty tube from the ellipses and their sampled boundary points, each ellipse needs to be aligned to a common reference direction (for more information on how this is performed, see section IV-A). This alignment is important for establishing correspondences between the ellipses and their boundary samples during the connection of vertices into triangles during the visualization step, see section C. If the alignment is not performed, the connections between ellipses can be out of order, and the tube can twist, as seen in fig. 7. During this alignment, a limitation is introduced based on the inclination of the asteroid's orbit. However, after further analysis of the currently known asteroid population, it was found that no asteroid with such an inclination exists, as seen in the plot in fig. 15. This plot was created by searching all asteroids (895,953 objects) listed at <https://minorplanetcenter.net/iau/MPCORB/MPCORB.DAT> as of July 1st, 2026. The most inclined object found was (518151) 2016 FH₁₃ with an inclination of approximately 86°. During the search and plotting, the inclinations were normalized to a range of 0–90°, where negative inclinations or inclinations above 90° (indicating retrograde orbits) were normalized.

E. Meta Data

To embed additional data that can later be used to color the side of the tube with a colormap, a list of data values is added for each ellipse boundary sample, such as the density value. The density value of each ellipse reflects the skewness of the distribution, calculated by adding up the distances from

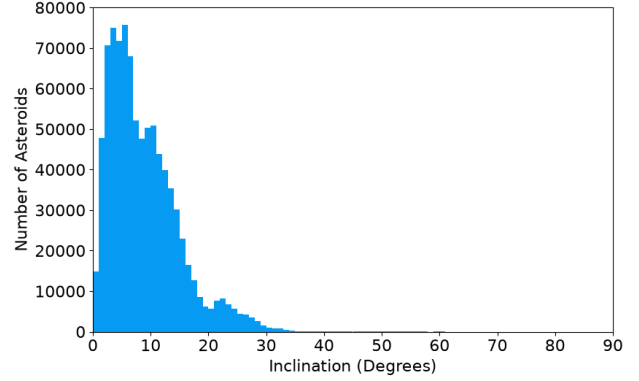


Fig. 15: Distribution of asteroids by inclination, showing the number of asteroids in each 1° bin from 0–90° as of July 1st, 2026, using data from minorplanetcenter.net/iau/MPCORB/MPCORB.DAT.

the ellipse sample to every variant on the plane. These values are then normalized and inverted, giving an estimation of the density of variants near that ellipse sample. Additionally, the timestamp for each ellipse is embedded in this data list to enable animation in the visualization.

For the cut-plane visualization, a texture is used to communicate additional data on the distribution of variants at each time step. In order to enable the storage of floating-point values in this texture that is later colorized using a colormap, we create a custom texture format for *NEOviz*. This texture format is simply a list of floating-point values per pixel and additional metadata about the value range in each channel.

Lastly, the ellipse sample points (or vertices) of the uncertainty tube, the textures, together with the additional metadata, are written to a file, which is read by the visualization system OpenSpace to render the uncertainty tube.

APPENDIX C VISUALIZATION

The visualization was integrated into the astrovisualization system OpenSpace using C++ and OpenGL on Windows. OpenSpace can be controlled using Lua scripts and the graphical user interface (GUI), see fig. 16. For more information, refer to the documentation page of OpenSpace available at <https://docs.openspaceproject.com/releases-v0.22/>.

The uncertainty tube is constructed in OpenSpace by connecting the vertices generated during the pre-processing step with triangles to form a mesh, see fig. 17 (a). This is done by connecting the vertices of one slice to the corresponding vertices of the slice for the next time step. The cut-plane is generated by using a center position of the cut-plane and constructing a triangle fan emanating from the center to the edge vertices of the current slice. For polygonal uncertainty tubes, the center position of the polygon shape is not trivial, and using the mean or median position of the distribution can be temporally unstable. Therefore, *NEOviz* uses the position of the best-fitting orbit at the same time as an approximation for the center position for polygonal slices. However, this is

not perfect, as the best-fitting orbit position may be located outside the boundary of the polygon, which can cause artifacts. Therefore, *NEOviz* uses elliptical slices for the uncertainty tube by default. If the current simulation time is between two time steps in the data, the two neighboring slices in time are interpolated. The normals for the triangles can be chosen by the user to either be smooth or flat, giving either smooth or hard shadows during shading, see fig. 17 (b). However, since the shading can interfere with colormap interpretation, the user can toggle the shading off in the GUI. To animate the uncertainty tube over time, the embedded timestamps on each ellipse are used to determine which part of the uncertainty tube should be visible at the current timestamp in OpenSpace. As the simulation time in OpenSpace progresses, more of the uncertainty tube reveals itself. To interpolate between time steps, linear interpolation is used for the uncertainty tube, while the user can choose between linear and nearest-neighbor interpolation for the cut-plane visualization.

The side of the uncertainty tube can show additional data using the embedded list of data from pre-processing. This includes whether the variant distribution is Gaussian, whether any deviation was detected, as well as the statistical standard deviation and variance in the major and minor axes of the ellipse. The user can interactively select which data to visualize and choose a colormap via the GUI, see <https://matplotlib.org/stable/users/explain/colors/colormaps.html> for all available colormaps. The cut-plane visualization has similar interactivity and uses the same colormaps. However, the cut-plane visualization can only be used for a limited number of time steps, since each time step is an OpenGL texture. To store the textures, an OpenGL texture array object is used, which has a maximum size that needs to be considered when choosing the time range to analyze.

To support analysis of individual variants, each variant trajectory can be added as a separate trail with the GUI

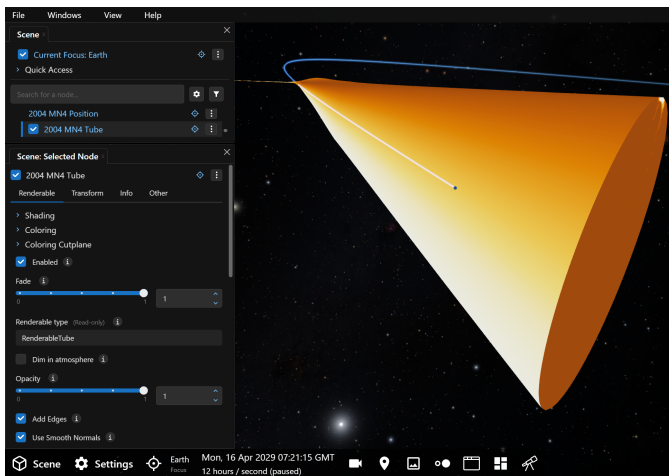
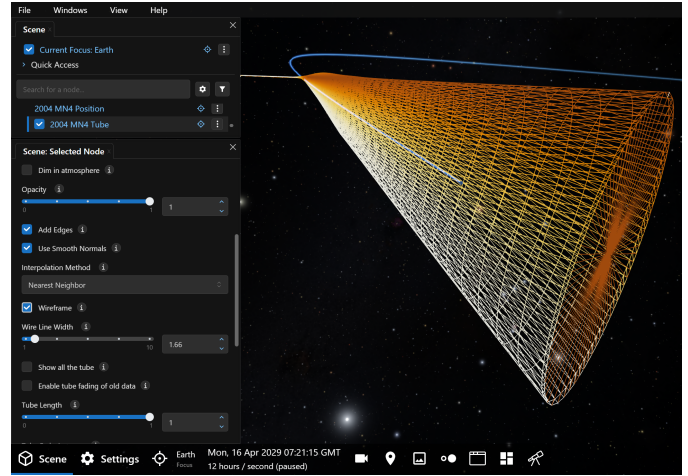
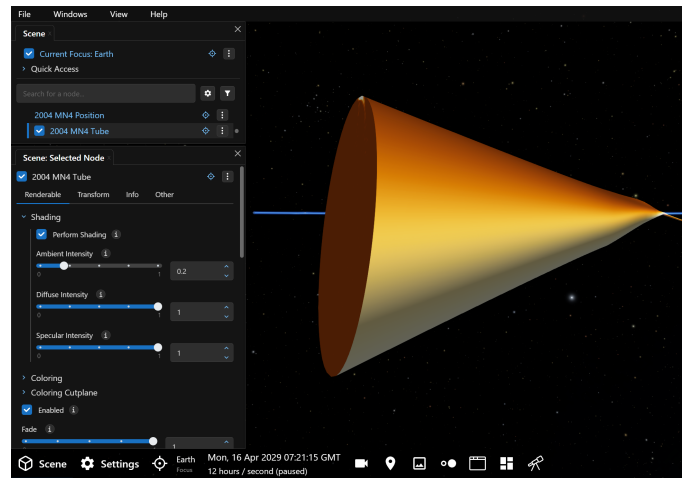


Fig. 16: The user interface in OpenSpace showing some of the settings for the uncertainty tube that the user can interact with. Here, the Apophis uncertainty tube is shown moments after the predicted impact in 2029, using historical data from Dec 27th, 2004.



(a) The uncertainty tube rendered as a wireframe



(b) The uncertainty tube with Phong shading

Fig. 17: Alternative rendering options for the uncertainty tube together with the graphical user interface in OpenSpace. In the background, the uncertainty tube of Apophis is shown moments after the predicted impact in 2029, using historical data from Dec 27th, 2004.

using the SPICE kernels generated in the pre-processing. By adding all variants in this manner, the previous spaghetti plot visualization can be rendered. The point cloud visualization can also be rendered in this way by adjusting the appearance of the trails to points. However, adding too many variants can slow down the system significantly, making interactive exploration difficult or impossible. To generate the example videos and images of the paper, the session recording feature of OpenSpace was used to render the material in an offline manner.

The impact map is generated with a task in OpenSpace using the generated SPICE kernels. This process was initially implemented in the Python pre-processing, but was transferred to C++ in order to speed up the computation. For each impact location found, a Gaussian dot is added onto a transparent equirectangular map of the Earth. Once all impact locations have been placed on the map, the result is normalized, and a

colormap is applied to produce the impact probability map. To create the impact risk probability map, a nightlight intensity map [59] is sampled in a region around the impact location, and the average intensity in this region is used to adjust the strength of the Gaussian dot placed on the map. The user can configure the Gaussian dots, such as their size, used to plot the impact locations and sample the nightlight map. The resulting maps are then layered on top of the existing Earth map in

OpenSpace to visualize them in context with the uncertainty tube, and the user can toggle between them with the GUI.

The uncertainty ellipsoid is constructed by deforming a standard wireframe sphere in OpenSpace. This sphere is scaled according to the pre-computed axis lengths of the ellipsoid, and then rotated and translated to its correct 3D location in space.